Introduction to Graph

CSE 4303: Data Structures
Lecture 7

Asaduzzaman Herok
Department of CSE
Islamic University of Technology

December 25, 2025

Recap: From Linear to Hierarchical

Before we introduce our new topic, let's review our journey so far:

- **Linear Data Structures** (Arrays, Linked Lists, Stacks, Queues):
  - Data is arranged sequentially.
  - Elements have a 1:1 relationship (one predecessor, one successor).
- **Hierarchical Data Structures** (Trees, BST):
  - Data is arranged in a hierarchy.
  - Elements have a 1:N relationship (Parent-Child).
  - *Constraint:* A tree cannot contain cycles. There is only one path between any two nodes.

Motivation: Limitations of Trees

Why do we need a new data structure?

- 👉 Trees are excellent for hierarchies (like file systems), but the real world is often more complex.
- 👉 **Scenario 1: A City Map**
  - You can drive from Point A to Point B and circle back to Point A. (A Cycle). Trees do not allow cycles.
- 👉 **Scenario 2: Social Networks**
  - Person A is friends with Person B, who is friends with Person C, who is also friends with Person A.
  - This "many-to-many" relationship cannot be modeled by a tree.

Conclusion: We need a structure that allows complex, non-hierarchical connections.

Introduction to Graphs

- 👉 A **Graph** is the most general non-linear data structure.
- 👉 It is a generalization of the tree structure.
- 👉 Instead of a strict parent-to-child relationship, a graph allows any kind of complex relationship between nodes.

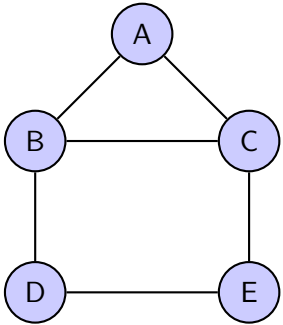
Key Concept

Graphs are used to model any situation where entities (things) are related to each other in pairs.

Formal Definition (Section 13.1)

Mathematically, a graph G is defined as an ordered set (V, E), where:

- **V(G)**: A finite, non-empty set of **vertices** (also called nodes).
- **E(G)**: A set of **edges** that connect pairs of these vertices.



Example

If we have a Graph G:

- $V(G) = \{A, B, C, D, E\}$
- $E(G)$ would contain pairs like $\{(A, B), (A, C), (D, E)...\}$ representing connections.

Real-World Analogy

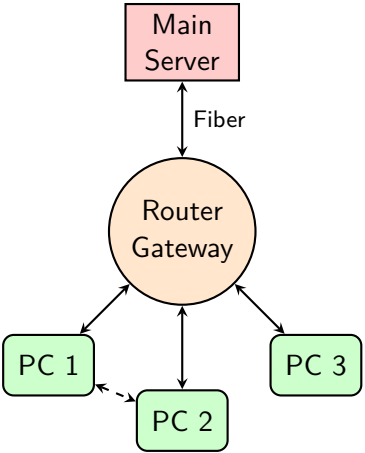
Graphs model "Pairwise Relationships":

👉 **Transportation Networks:**

- *Vertices:* Airports, Intersections, or Ports.
- *Edges:* Airline flights, Roads, or Shipping routes.

👉 **Communication Networks:**

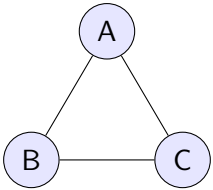
- *Vertices:* Computers or Routers.
- *Edges:* Cables or Wireless links.



Terminology: Neighbors, Degree, and Regularity

1. **Adjacent Nodes (Neighbors)**
   - 👉 If edge $e = (u, v)$ connects nodes u and v, they are **adjacent**.
2. **Degree of a Node ($deg(u)$)**
   - 👉 Total number of edges connected to node u.
   - 👉 **Isolated Node:** A node with $deg(u) = 0$.
3. **Regular Graph**
   - 👉 A graph where every vertex has the **same degree**.
   - 👉 *k-regular:* Every node has degree k.

Isolated
deg(D) $= 0$



2-Regular Graph
(All degrees $= 2$)

Terminology: Paths and Cycles

4. Path

👉 A sequence of nodes $P = \{v_0, v_1, \ldots, v_n\}$ where each consecutive pair is adjacent.

5. Closed Path

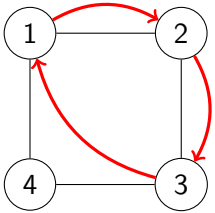
👉 A path where the start and end nodes are the same ($v_0 = v_n$).

6. Simple Path

👉 A path where all nodes are distinct (except possibly start/end for a closed simple path).

7. Cycle

👉 A path where the first and last vertices are the same but no repeated vertices or edges.



Cycle: $1 \to 2 \to 3 \to 1$

(Closed Path)

Terminology: Connectivity Types

8. Connected Graph

- A path exists between **any** two pairs of vertices.

9. Complete Graph

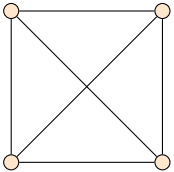
- Every node is connected to every other node.
- Has $n(n-1)/2$ edges.

10. Clique

- A subset of vertices where every two distinct vertices are connected.

11. Tree

- A connected graph with no cycles.



Complete Graph (K_4)

(Also a Clique of size 4)

Terminology: Edge Properties & Size

12. Weighted / Labelled Graph

- **Labelled**: Edges are assigned data.
- **Weighted**: Edges are assigned a value/cost $w(e)$ (e.g., distance).

13. Loop

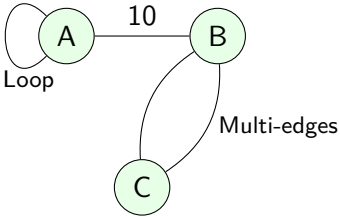
- An edge that connects a node to itself (u, u).

14. Multi-graph

- Contains **Multiple Edges** (parallel edges between same nodes) or Loops.

15. Size of a Graph

- The total number of edges in the graph.



Weighted Multi-graph

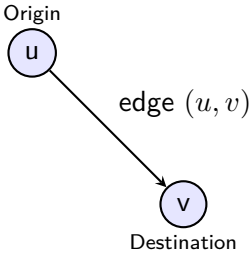
Section 13.3: Directed Graphs (Digraphs)

Definition

- A **Directed Graph** (Digraph) G is a graph where every edge has a specific direction.
- Edges are **ordered pairs** (u, v).

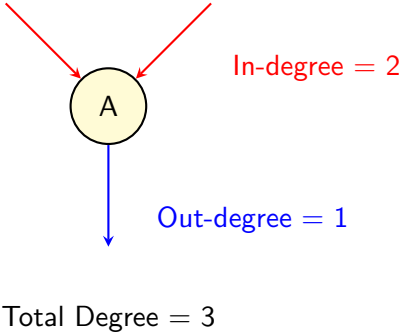
Edge Properties

- For an edge $e = (u, v)$:
  - **Origin/Initial Point**: Node u (where edge begins).
  - **Destination/Terminal Point**: Node v (where edge ends).
  - **Predecessor**: u is the predecessor of v.
  - **Successor**: v is the successor of u.



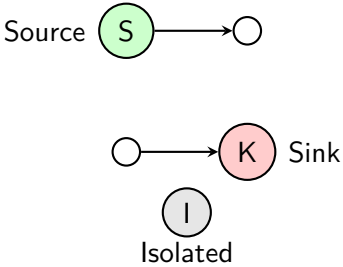
Terminology: In-Degree and Out-Degree

1. **Out-Degree ($outdeg(u)$)**
   - 👉 Number of edges **originating** at u.
2. **In-Degree ($indeg(u)$)**
   - 👉 Number of edges **terminating** at u.
3. **Total Degree ($deg(u)$)**
   - 👉 $deg(u) = indeg(u) + outdeg(u)$.



Terminology: Types of Vertices

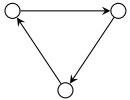
1. **Source**: Node with positive out-degree but **zero in-degree**.
2. **Sink**: Node with positive in-degree but **zero out-degree**.
3. **Isolated Vertex**: Degree is zero ($indeg = outdeg = 0$).
4. **Pendant (Leaf) Vertex**: Degree is one.
5. **Cut Vertex**: Deletion disconnects the graph.



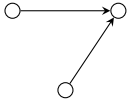
Connectivity in Directed Graphs

Reachability: Node v is reachable from u if a directed path exists $u \to \cdots \to v$.

1. **Strongly Connected**:
   - 👉 Path exists between **every pair** in both directions ($u \to v$ AND $v \to u$).
2. **Unilaterally Connected**:
   - 👉 Path exists between any pair in at least **one direction** ($u \to v$ OR $v \to u$).
3. **Weakly Connected**:
   - 👉 Connected only if direction is ignored (underlying undirected graph is connected).



Strongly Connected



Weakly Connected
(No path between X and Z)

Simple Directed Graphs vs Multi-Graphs

Parallel/Multiple Edges

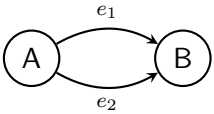
- Distinct edges connecting the same endpoints ($u \to v$ twice).

Simple Directed Graph

- Has **no parallel edges**.
- *Note:* It may contain cycles and loops (max 1 loop per node).

Directed Multi-Graph

- Contains parallel edges.



Multi-Graph
(Parallel Edges)

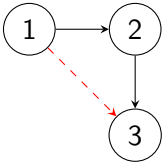
Transitive Closure (Section 13.3.2)

Definition (G^*)

- Given a graph $G = (V, E)$, the Transitive Closure $G^* = (V, E^*)$.
- An edge (u, v) exists in G^* if and only if there is a valid **path** from u to v in original graph G.

Why is it needed?

- To answer **Reachability Questions** in $O(1)$ time.
- "Can we get from A to B?" becomes "Is there an edge (A, B) in G^*?"
- Used in database queries and compiler construction.



Red dashed edge added in G^*

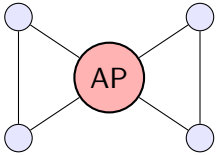
Section 13.4: Articulation Points

Definition

- An **Articulation Point** (or Cut Vertex) is a vertex v such that if we remove v and all edges incident to it, the graph becomes disconnected.
- Removing it increases the number of connected components.

Significance

- Represent "Single Points of Failure" in a network.
- If a router acting as an articulation point fails, the network splits.



Removing AP disconnects left & right

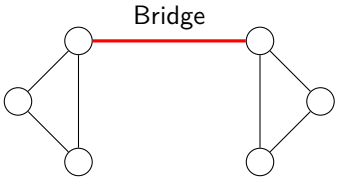
Bridges (Cut Edges)

Definition

- A **Bridge** is an edge e such that its removal results in a disconnected graph.
- Bridges cannot lie on a cycle.

Relationship with Cycles

- If an edge is part of a cycle, there is an alternative path between its endpoints, so removing it does not disconnect the graph.
- Bridges are critical links for connectivity.



Removing this edge splits the graph

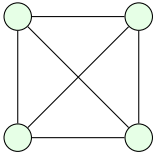
Bi-connected Graphs

Bi-connected Graph

- 👉 A graph that contains **no articulation points**.
- 👉 It cannot be disconnected by removing a single vertex.

Properties

- 👉 There are at least two vertex-disjoint paths between any pair of vertices.
- 👉 Highly robust: If one node fails, the rest remain connected.



Bi-connected
(No single point of failure)

Section 13.5: Graph Representation

There are three common ways to store graphs in computer memory:

1. **Sequential Representation** (Adjacency Matrix): Uses a 2D array.
2. **Linked Representation** (Adjacency List): Uses a linked list of neighbors.
3. **Adjacency Multi-list**: An extension of the linked representation for edge-based storage.

Choice of Representation

The choice depends on the type of graph (sparse vs. dense) and the operations to be performed.

13.5.1 Adjacency Matrix

Definition

- An $n \times n$ 2D array (or bit/Boolean matrix).
- Rows and columns are labeled by graph vertices.

Entries (A_{ij})

- $A_{ij} = 1$ if there is an edge from vertex i to vertex j
- $A_{ij} = 0$ if there is no edge.

Properties

- Memory usage: $O(n^2)$.
- **Symmetric** for undirected graph.
- **Weighted Graphs**: $A_{ij} = weight$ instead of 1.

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

Adjacency Matrix for a sample graph

Adjacency Matrix: Pros and Cons

Advantages

- Simple to implement.
- Determining if an edge exists between two nodes is $O(1)$.

Disadvantages

- **Space Inefficient**: Consumes $O(n^2)$ memory even if the graph is sparse (few edges).
- **Hard to Resize**: Adding a new node requires resizing the entire matrix.

Power of Matrix (A^k)

An entry in row i, column j of matrix A^k gives the number of paths of length k from node v_i to v_j.

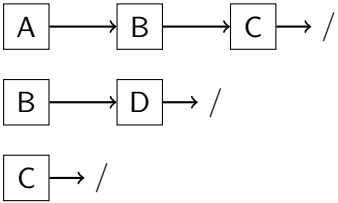
13.5.2 Adjacency List

Concept

- An array of pointers (Head nodes), where each entry points to a linked list.
- The linked list for node i contains all nodes adjacent to i.

Memory Efficiency

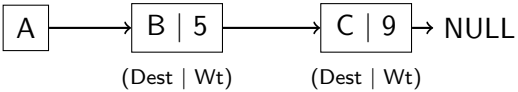
- Preferred for **Sparse Graphs**.
- Memory depends on edges: $O(V + E)$ (Directed) or $O(V + 2E)$ (Undirected).



Adjacency List Structure

Adjacency List for Weighted Graphs

- Adjacency lists can be modified to store weighted graphs.
- Each node in the linked list stores two values:
  - The **destination vertex**.
  - The **weight/cost** of the edge.



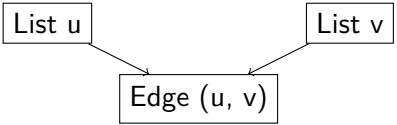
13.5.3 Adjacency Multi-list

Overview

- An edge-based representation (vs. vertex-based).
- Useful when edges need to be processed directly or marked (e.g., "visited").

Structure

- **Directory of Nodes**: Points to edge lists.
- **Edge Nodes**: Each node represents an edge (u, v) and is part of two lists:
  - The list for node u.
  - The list for node v.
- Nodes are shared among lists.



Edge node shared by lists

Section 13.6: Graph Traversal Algorithms

What is Graph Traversal?

Graph traversal is the process of visiting all the vertices of a graph in a systematic order.

Three States of a Node During traversal, a node can be in one of three states:

1. **STATUS = 1 (Ready)**: The initial state. The node has not yet been visited or added to the waiting list.
2. **STATUS = 2 (Waiting)**: The node has been added to the queue (or stack) but not yet processed.
3. **STATUS = 3 (Processed)**: The node has been visited and its neighbors have been added to the waiting list.

13.6.1 Breadth-First Search (BFS)

Algorithm Concept

- 👉 Starts at a source node and explores all its immediate neighbors first.
- 👉 Then moves to the neighbors of those neighbors.
- 👉 Moves **level by level** (Level 0 $\rightarrow$ Level 1 $\rightarrow$ Level 2...).

Data Structure Used

- 👉 **Queue** (FIFO: First In, First Out).
- 👉 Standard approach for finding the *Shortest Path* in unweighted graphs.

Algorithm: Breadth-First Search (BFS) I

Data Structure Required

A **Queue** (FIFO) is used to store nodes that are waiting to be processed.

Algorithmic Steps:

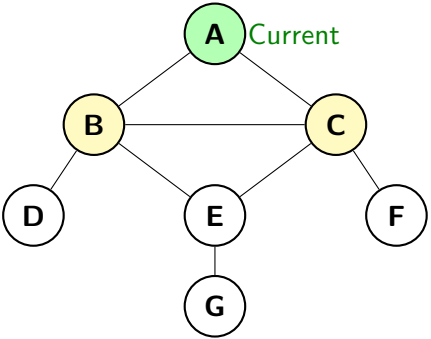
1. **Initialize**: Set all nodes to state `READY` (Unvisited). Initialize an empty Queue Q.
2. **Start**:
   - Select the starting node S.
   - Change state of S to `WAITING`.
   - Enqueue S into Q.
3. **Loop** (Repeat while Q is not empty):
   - Dequeue the front node u.
   - Process u (Print or Store). Change state to `PROCESSED`.

Algorithm: Breadth-First Search (BFS) II

- For each neighbor v of u:
  - ⋆ If state of v is `READY`:
  - ⋆ Mark v as `WAITING` (Visited).
  - ⋆ Enqueue v.

4. **End**: Stop when Q is empty.

BFS Execution Steps

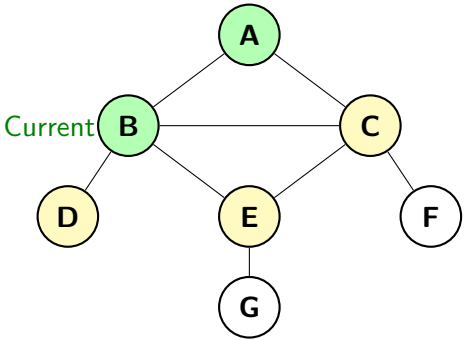


Step 1:
Start at A. Process A.
Enqueue **B, C**.

Queue:

← Out | B | C | In ←

BFS Execution Steps

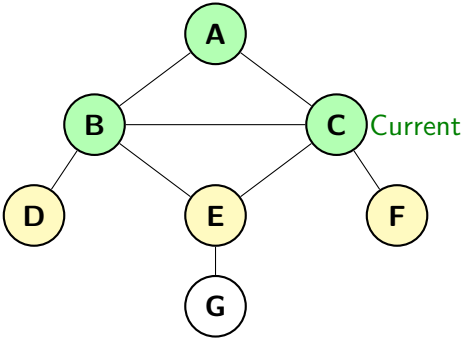


Step 2:
Dequeue B.
Enqueue **D, E**. (C skipped).

Queue:

← Out | C | D | E | In ←

BFS Execution Steps



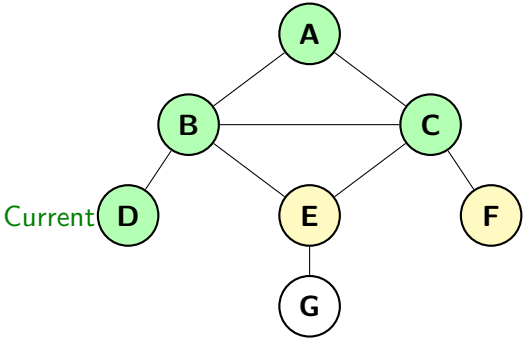
Step 3:
Dequeue C.
Enqueue **F**. (E skipped).

Queue:

D	E	F

← Out In ←

BFS Execution Steps



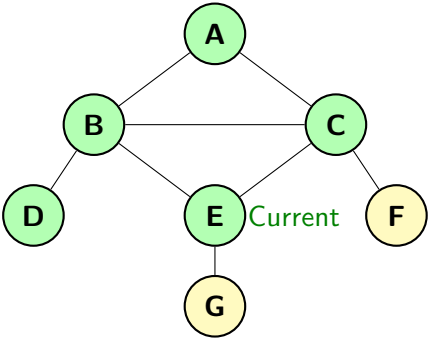
Step 4:
Dequeue D.
No new nodes.

Queue:

E	F

← Out In ←

BFS Execution Steps



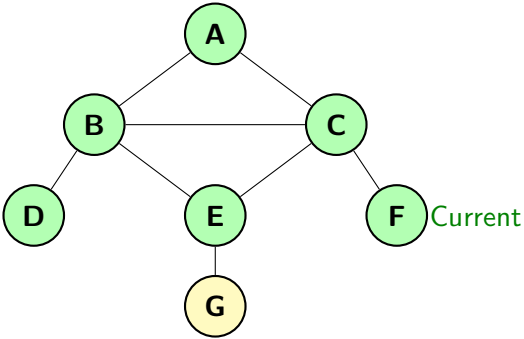
Step 5:
Dequeue E.
Enqueue **G**.

Queue:

F	G

← Out In ←

BFS Execution Steps



Step 6:
Dequeue F.
No new nodes.

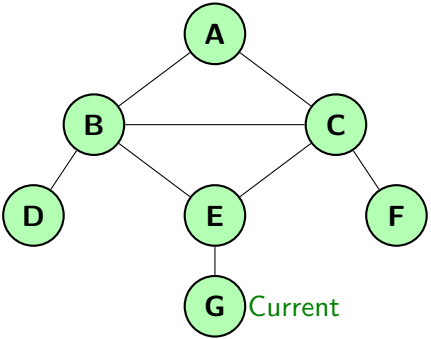
Queue:

G

← Out

In ←

BFS Execution Steps



Step 7:
Dequeue G.
Done.

Queue:

Empty

← Out In ←

Complexity Analysis: Breadth-First Search (BFS) I

Time Complexity

The time depends heavily on the graph representation used.

- **Adjacency List:** $O(V + E)$
  - **Vertex Operations:** Every vertex is Enqueued and Dequeued exactly once $\rightarrow O(V)$.
  - **Edge Operations:** For every vertex, we iterate through its list of adjacent edges. Across the whole run, we visit every edge once (directed) or twice (undirected) $\rightarrow O(E)$.
- **Adjacency Matrix:** $O(V^2)$
  - For every vertex dequeued, we must iterate through its entire row of length V to find connected neighbors.
  - Total operations: V vertices $\times$ V checks $\rightarrow O(V^2)$.

Complexity Analysis: Breadth-First Search (BFS) II

Space Complexity: $O(V)$

- **Queue:** In the worst case (e.g., a Star Graph), the queue might hold up to $V - 1$ vertices at once.
- **Visited Array:** Stores status for V nodes.

13.6.2 Depth-First Search (DFS)

Algorithm Concept

- 👉 Starts at a source node and explores as far as possible along each branch before backtracking.
- 👉 Goes "deep" into the graph.

Data Structure Used

- 👉 **Stack** (LIFO: Last In, First Out).
- 👉 Often implemented using **Recursion** (system call stack).

Algorithm: Depth-First Search (DFS) I

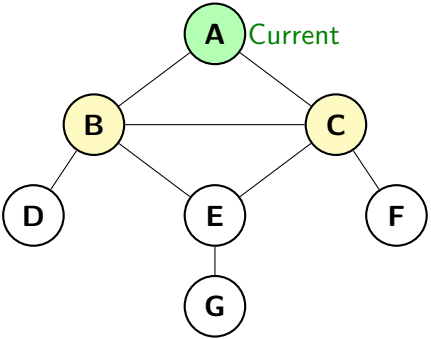
Data Structure Required

A **Stack** (LIFO) is used to backtrack once a path hits a dead end. This can be implemented explicitly or via **Recursion**.

Algorithmic Steps (Iterative approach):

1. **Initialize**: Set all nodes to `READY`. Initialize an empty Stack S.
2. **Start**:
   - Push the starting node v onto S.
3. **Loop** (Repeat while S is not empty):
   - Pop the top node u from S.
   - **If** u is not yet `PROCESSED`:
     - Process u and mark as `PROCESSED`.
     - Push all `READY` neighbors of u onto S.
4. **End**: Stop when S is empty.

DFS Execution Steps



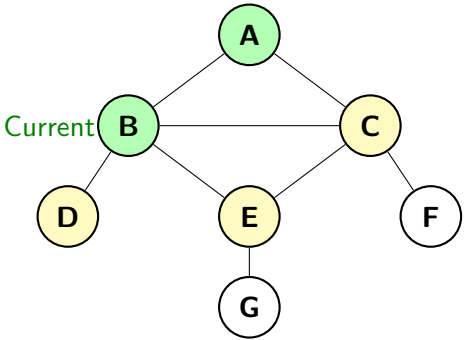
Step 1:
Start **A**. Push **C, B**.

Stack:

B
C

↑ Top

DFS Execution Steps



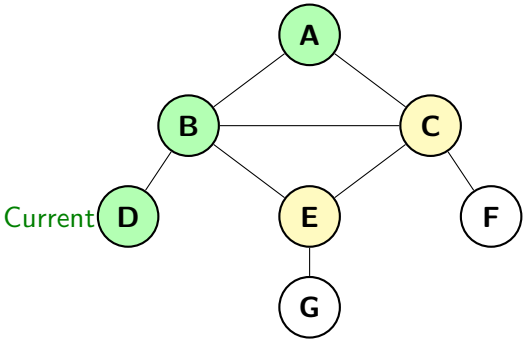
Step 2:
Pop **B**. Push **E, D**.

Stack:

D
E
C

↑ Top

DFS Execution Steps



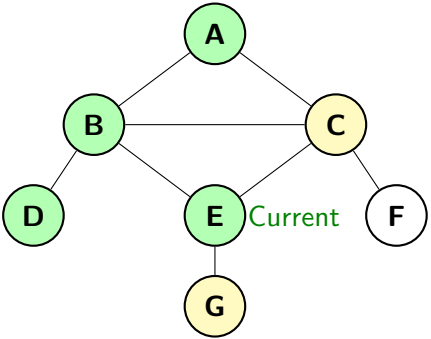
Step 3:
Pop **D**. Dead end.

Stack:

E
C

↑ Top

DFS Execution Steps



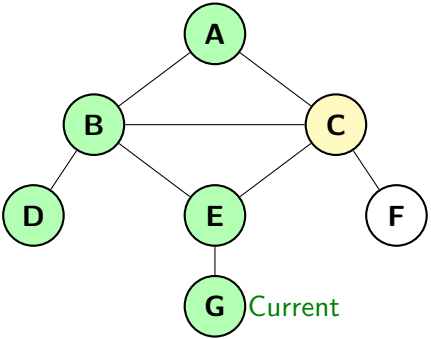
Step 4:
Pop **E**. Push **G**.

Stack:

G
C

↑ Top

DFS Execution Steps



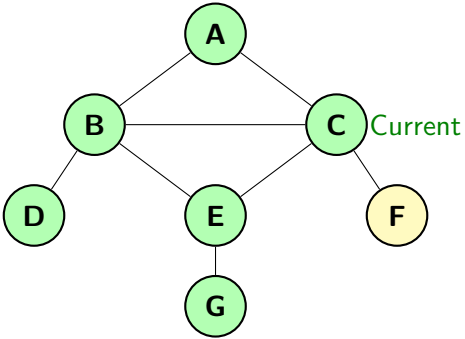
Step 5:
Pop **G**. Backtrack.

Stack:

C

↑ Top

DFS Execution Steps



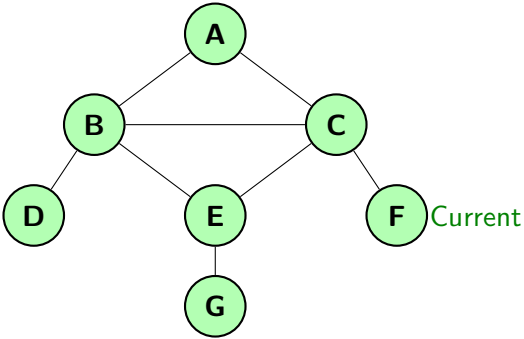
Step 6:
Pop **C**. Push **F**.

Stack:

F

↑ Top

DFS Execution Steps



Step 7:
Pop **F**. Done.

Stack:

Empty

↑ Top

Complexity Analysis: Depth-First Search (DFS) I

Time Complexity

Similar to BFS, the structure determines the efficiency.

- **Adjacency List:** $O(V + E)$
  - **Visits:** We enter the recursive function (or push to stack) for each vertex exactly once $\rightarrow O(V)$.
  - **Traversals:** Inside each visit, we iterate over the neighbors. The sum of all adjacency list lengths is proportional to the number of edges $\rightarrow O(E)$.
- **Adjacency Matrix:** $O(V^2)$
  - For every node visited, we must scan the entire row (all V columns) to identify valid edges.
  - Total operations: $V \times V \rightarrow O(V^2)$.

Complexity Analysis: Depth-First Search (DFS) II

Space Complexity: $O(V)$

- **Stack (Recursion Stack):**
  - **Worst Case:** $O(V)$ if the graph is a long line (Skewed Tree).
  - **Best Case:** $O(\log V)$ if the graph is a balanced tree structure.
- **Visited Array:** $O(V)$ to keep track of visited nodes.

13.7 Topological Sorting

Definition

Topological sorting is a linear ordering of vertices in a directed graph such that for every directed edge $u \to v$, vertex u comes before v in the ordering.

Key Constraints:

- 👉 The graph must be a **DAG** (Directed Acyclic Graph).
- 👉 If the graph contains a cycle, a topological sort is **impossible**.

Real-World Analogy:

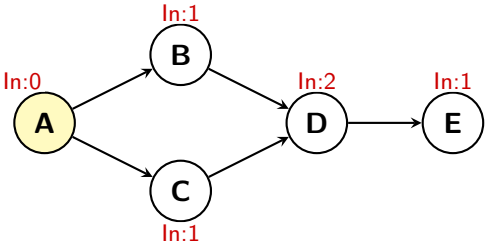
- 👉 **Prerequisites:** You cannot take "Advanced Data Structures" (v) before taking "Intro to C" (u).
- 👉 **Assembly:** You must put on socks (u) before shoes (v).

Algorithm Steps (Source Removal Method)

This approach uses **In-Degrees** to determine dependency.

1. **Calculate In-Degree**: Count the number of incoming edges for every node in the graph.
2. **Initialize Queue**: Add all nodes with `In-Degree = 0` to a Queue. (These are nodes with no dependencies).
3. **Process Loop** (While Queue is not empty):
   - Dequeue a node u.
   - Add u to the **Sorted Output List**.
   - For every neighbor v of u (effectively removing edge $u \to v$):
     - Decrement the In-Degree of v by 1.
     - **If** In-Degree of v becomes 0 $\to$ Enqueue v.
4. **Completion**: If the Sorted List contains all nodes, we are done. If not, the graph has a cycle.

Topological Sort Animation



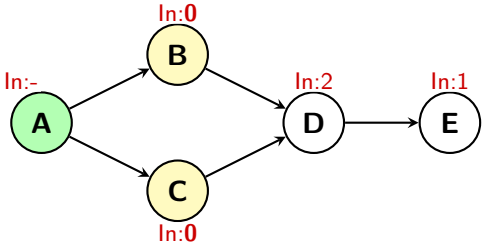
Log:
Step 1: Initialize.
Calc In-degrees.
A has 0 in-degree → Enqueue.

Queue:

A

Sorted Output List:
`{ (Empty) }`

Topological Sort Animation



Log:

Step 2: Process A.

Add A to Output.

Reduce neighbors B & C.

B, C become 0 → Enqueue.

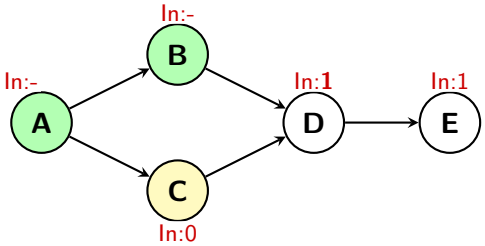
Queue:

B	C

Sorted Output List:

`{ A }`

Topological Sort Animation



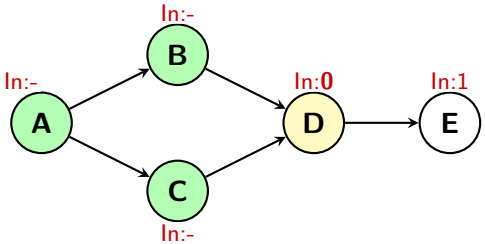
Log:
Step 3: Process B.
Add B to Output.
Reduce neighbor D ($2 \to 1$).
D is not yet 0.

Queue:

C

Sorted Output List:
`{ A, B }`

Topological Sort Animation



Log:

Step 4: Process C.

Add C to Output.

Reduce neighbor D ($1 \to 0$).

D becomes 0 $\to$ Enqueue.

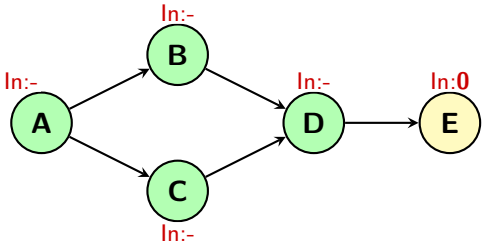
Queue:

D

Sorted Output List:

`{ A, B, C }`

Topological Sort Animation



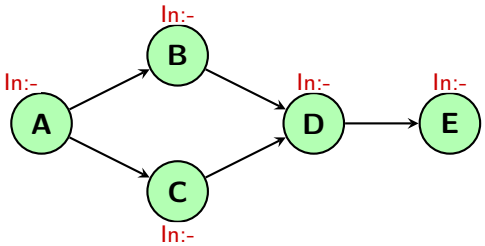
Log:
Step 5: Process D.
Add D to Output.
Reduce neighbor E ($1 \to 0$).
E becomes 0 $\to$ Enqueue.

Queue:

E

Sorted Output List:
`{ A, B, C, D }`

Topological Sort Animation



Log:
Step 6: Process E.
Add E to Output.
Queue Empty → Done.

Queue:

Empty

Sorted Output List:
`{ A, B, C, D, E }`

Complexity Analysis: Time Complexity $O(V + E)$ I

The efficiency comes from avoiding repeated scans of the node list.

1. **Step 1: Calculate In-Degrees** $O(V + E)$
   - We iterate through every node (V) and every outgoing edge (E) once to build the initial count.
2. **Step 2: Initial Discovery (Static Scan)** $O(V)$
   - We scan the In-Degree array **once** to find all nodes starting with 0 dependencies and add them to the Queue.

Complexity Analysis: Time Complexity $O(V + E)$ II

3. **Step 3: Processing Loop (Dynamic Update)** $O(E)$
   - **The mechanism:** We do **not** rescan all nodes to find the next available task.
   - Instead, we find them **dynamically**:
     - ⋆ When processing node u, we visit only its neighbors v.
     - ⋆ We decrement `InDegree[v]`.
     - ⋆ **Immediate Check:** If `InDegree[v]` drops to 0, we enqueue v immediately.
   - Since every edge is visited exactly once for a decrement operation, this entire phase takes linear time relative to edges.

Total Time: $O(V + E) + O(V) + O(E) \approx \mathbf{O(V + E)}$

Complexity Analysis: Space & Correctness I

Space Complexity: $O(V)$

We need auxiliary storage proportional to the number of vertices:

- 👉 **In-Degree Array:** Stores an integer count for all V nodes $\rightarrow O(V)$.
- 👉 **Queue:** In the worst case (e.g., a graph with no edges), all V nodes are stored in the queue simultaneously $\rightarrow O(V)$.
- 👉 **Sorted Output List:** Stores the result $\rightarrow O(V)$.

Note: The graph storage itself (Adjacency List) takes $O(V + E)$, but that is input storage.

Complexity Analysis: Space & Correctness II

Cycle Detection (Correctness)

Topological Sort doubles as a Cycle Detection algorithm.

- 👉 We maintain a counter of processed nodes.
- 👉 **Logic:** If `count < V` when the Queue becomes empty, it implies a **Cycle** exists.
- 👉 **Why?** Nodes in a cycle never reach In-Degree 0, so they never enter the Queue.

🤝 Reference & Q&A

Reference: Data Structures c Using 2nd Edition, Reema Thareja

👉 **Chapter 13**:(Covered topics in the slides only)

Thank You!

👋 Thank you for your time and attention. 👋

🗨 Any questions or clarifications. 💭